



ESTRUTURAS DE DADOS DINÂMICA

Uma nota ruim não define quem você é nem o quanto pode alcançar, ela é apenas o retrato de um momento, não da sua capacidade.

Aprenda com ela, cresça com ela e mostre, na próxima oportunidade, a verdadeira força que você carrega.

AOWWWW



ESTRUTURAS DE DADOS DINÂMICA

São formas de organizar e armazenar dados em um programa de computador de maneira **flexível**, permitindo que o tamanho dos dados possa ser alterado durante a execução do programa.

ESTRUTURAS DE DADOS DINÂMICA

Segundo Tenenbaum (1995), estrutura de dados dinâmica refere-se aos conceitos e técnicas de organização e manipulação de dados em sistemas computacionais.

- Estruturas de Dados Dinâmicas
 - *Pilhas*
 - *Filas*
 - *Listas*
 - *Árvores*
 - *Grafos*



Por que estruturas
de dados
dinâmicas?



Ana
Moreno
Cidão
João

vetor

7	5	6	11	9	88
4	8	1	2	40	71
0	14	77	21	88	54
23	5	31	29	3	22

matriz

Vetores e Matrizes possuem
tamanhos fixos



Por que estruturas
de dados
dinâmicas?

Ana
Moreno
Cidão
João

vetor

7	5	6	11	9	88
4	8	1	2	40	71
0	14	77	21	88	54
23	5	31	29	3	22

matriz

Uma estrutura de dados dinâmica **pode crescer e/ou diminuir** enquanto novos elementos são inseridos/removidos.



Quais estruturas de dados dinâmicas estudaremos?

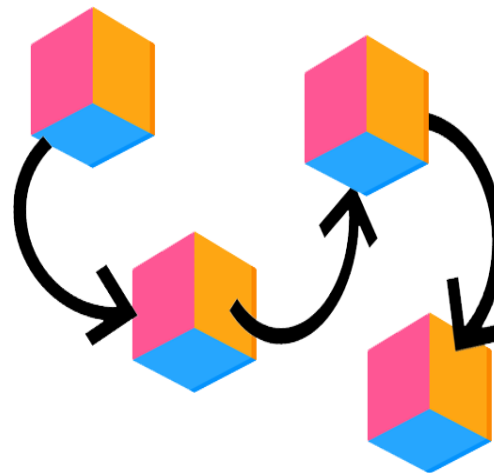
Pilhas



Filas



Listas encadeadas



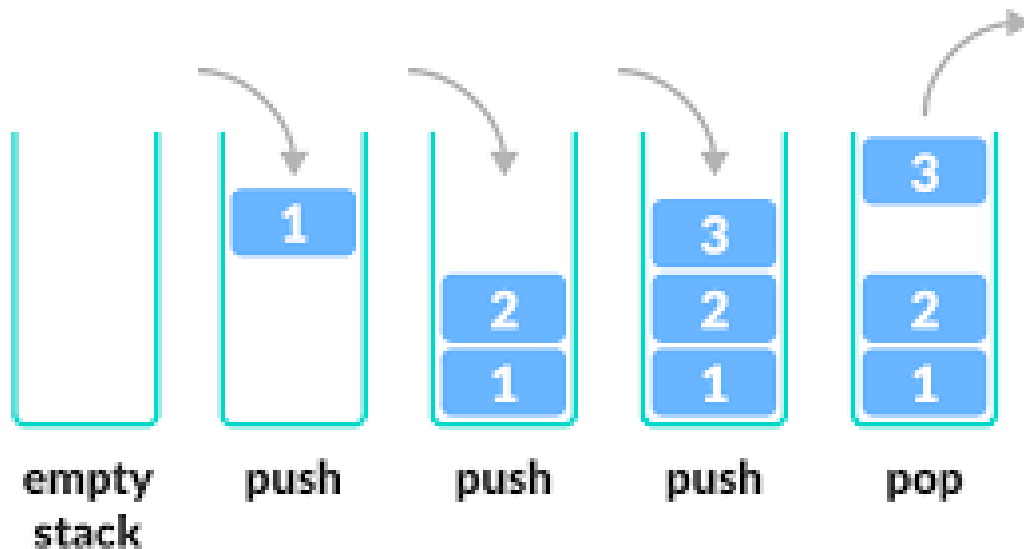
- *simplesmente encadeadas*
- *duplamente encadeadas*
- *listas circulares*



LIFO - Last In First Out

Pilha

Pilha



Estrutura de dados Pilha

- Simples
- Possui apenas duas operações:
 - Empilhar (**push**)
 - Desempilhar (**pop**)
- O último a entrar é o primeiro a sair (**LIFO** - *Last-in First-out*)

A operação empilhar e/ou desempilhar será sempre realizada no top

Como funciona
uma estrutura de
dados dinâmica?



Ao inserir um **novo** dado, uma nova região de memória é alocada e “**ligada**” na estrutura de dados por meio de **ponteiros**.

Como funciona a
inserção (empilhar)?

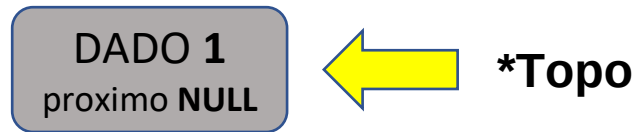


Uma estrutura de dados dinâmica possui ao menos duas partes:

- um campo para o dado
- um campo para um ponteiro para uma estrutura do mesmo tipo (**nó**)

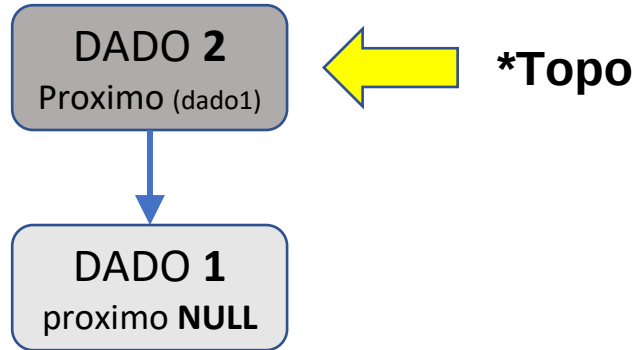
NULL ← *Topo

Lembre-se que sempre estaremos trabalhando com **ponteiros** e inicialmente teremos um ponteiro apontando para o topo da lista. No início o valor desse ponteiro será nulo (NULL) pois a lista estará vazia.

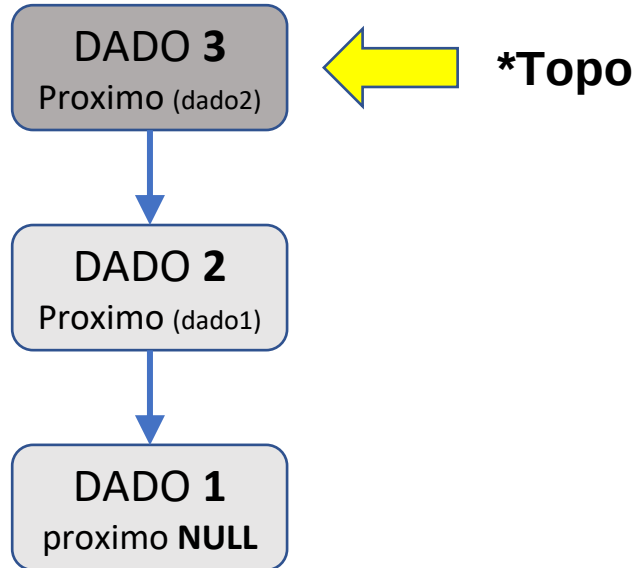


Ao inserir um elemento - que consiste em dado (que pode ser inteiro, real, caractere, vetor, estrutura, etc) e um ponteiro (que apontará para o próximo nó, que nesse caso será NULL)

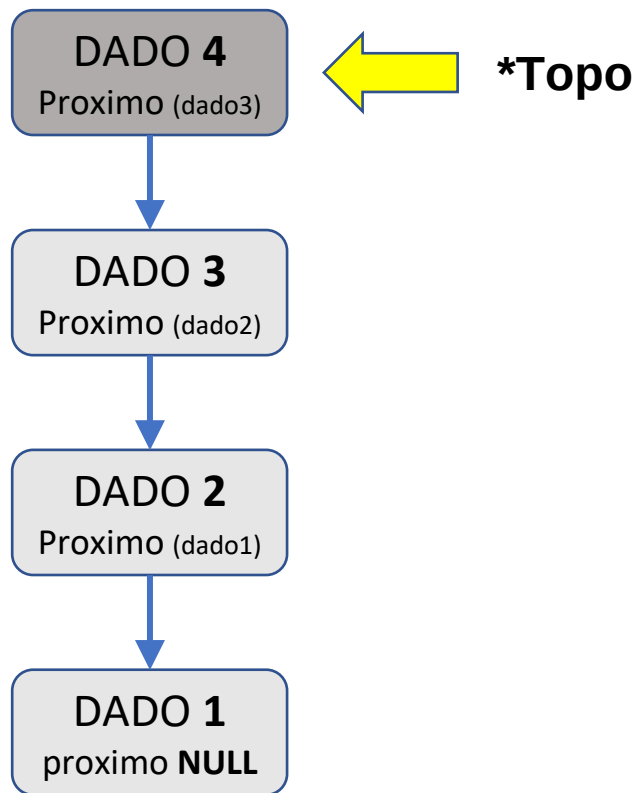
Cada **nó** possuirá um campo de informação e um ponteiro)



Ao fazer uma nova inserção, o **Topo** apontará para esse novo nó, e esse nó apontará para o elemento anterior (primeiro elemento)



Ao fazer uma nova inserção, o **Topo** apontará para esse novo nó, e esse nó apontará para o elemento anterior (DADO 2).

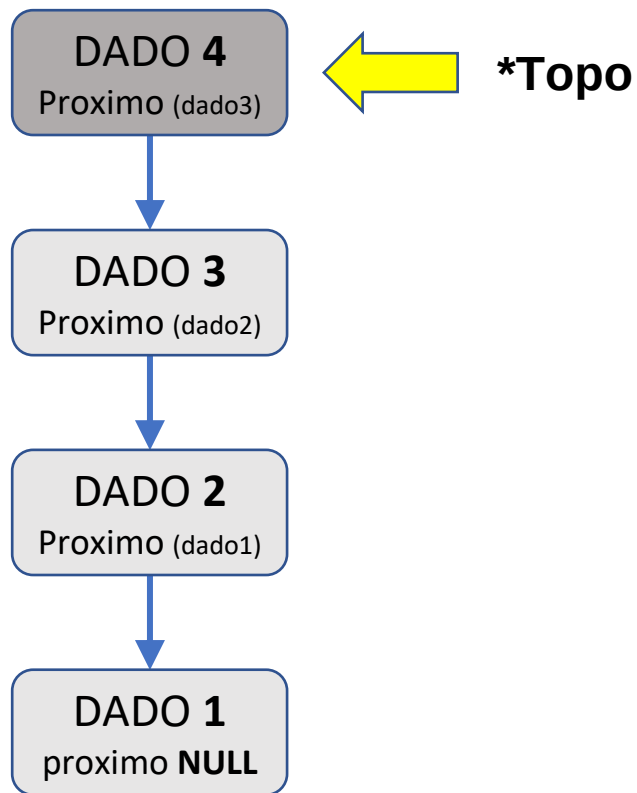


Ao fazer uma nova inserção, o **Topo** apontará para esse novo nó, e esse nó apontará para o elemento anterior (DADO 3)

Desta forma, o dado mais recente estará no topo da pilha.

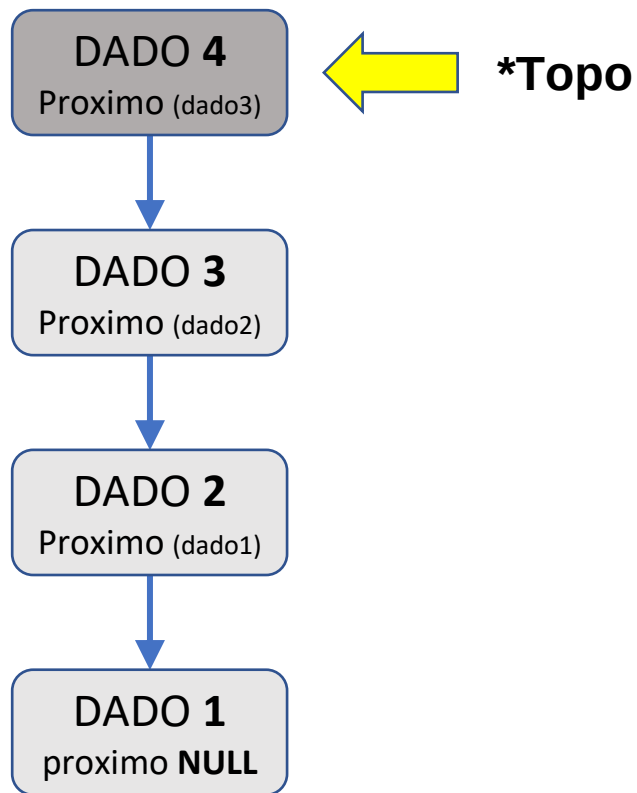
Como funciona a
remoção (desempilhar)?





A remoção possui 3 etapas:

1. *remove
2. Mudar o topo de lugar
3. Liberar a memória (free)

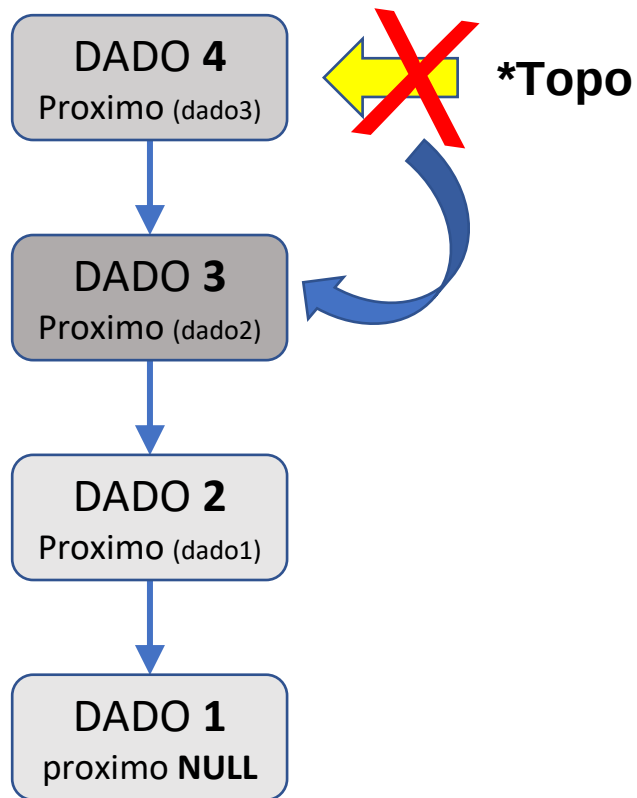


1) ***remove**

Criar um ponteiro auxiliar que irá apontar para o topo

A remoção possui 3 etapas:

1. ***remove**
2. Mudar o topo de lugar
3. Liberar a memória (free)

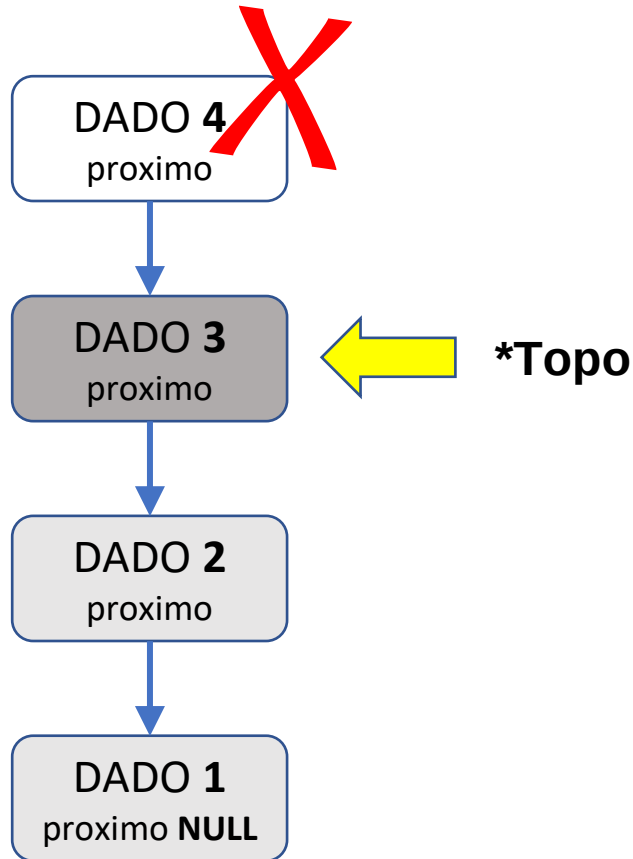


2) Mudar o topo de lugar

O *topo deverá apontar para o elemento abaixo.

A remoção possui 3 etapas:

1. *remove
2. **Mudar o topo de lugar**
3. Liberar a memória (free)

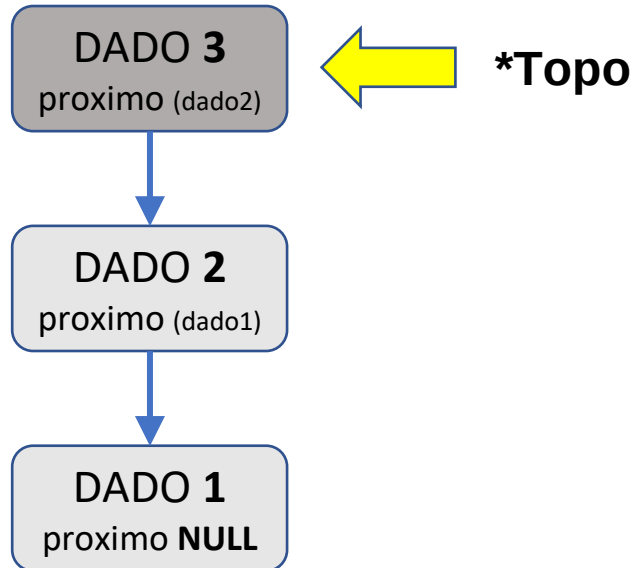


3) Liberar a memória

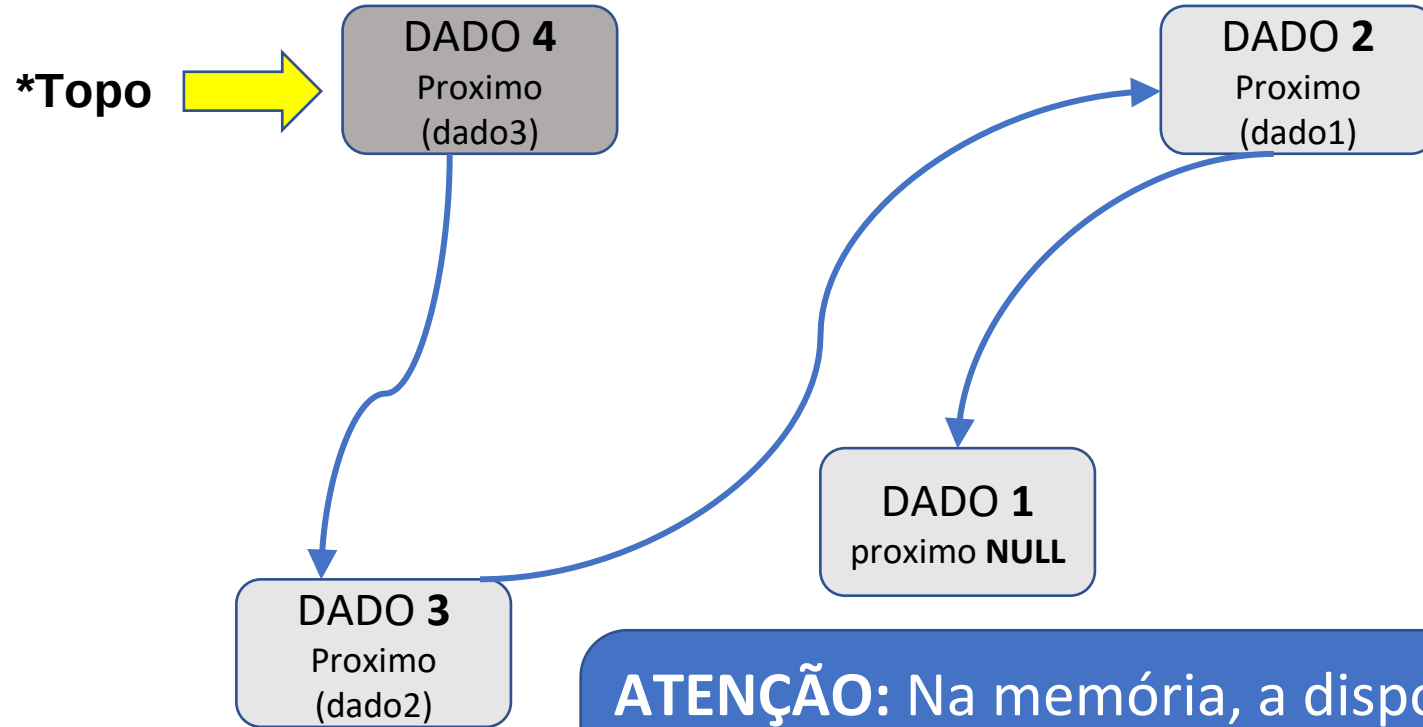
Liberar a memória do ponteiro auxiliar através do comando `free()`.

A remoção possui 3 etapas:

1. `*remover`
2. Mudar o topo de lugar
3. **Liberar a memória (`free`)**



Elemento removido!



ATENÇÃO: Na memória, a disposição dos **nós** ficarão dispersos (e não organizados como nas ilustrações anteriores)



**Implementando em
linguagem C**



Como seria
um nó da
nossa pilha?

Te fez lembrar
recursividade?

“no” é um apelido para que
possamos referenciá-lo
dentro da própria struct

```
typedef struct no{  
    int dado;  
    struct no *proximo;  
}No;
```

No exemplo acima, cada nó possui um dado do
tipo inteiro e um ponteiro do tipo nó para
apontar para próximo nó, lembrando que o
primeiro será sempre **NULL**.

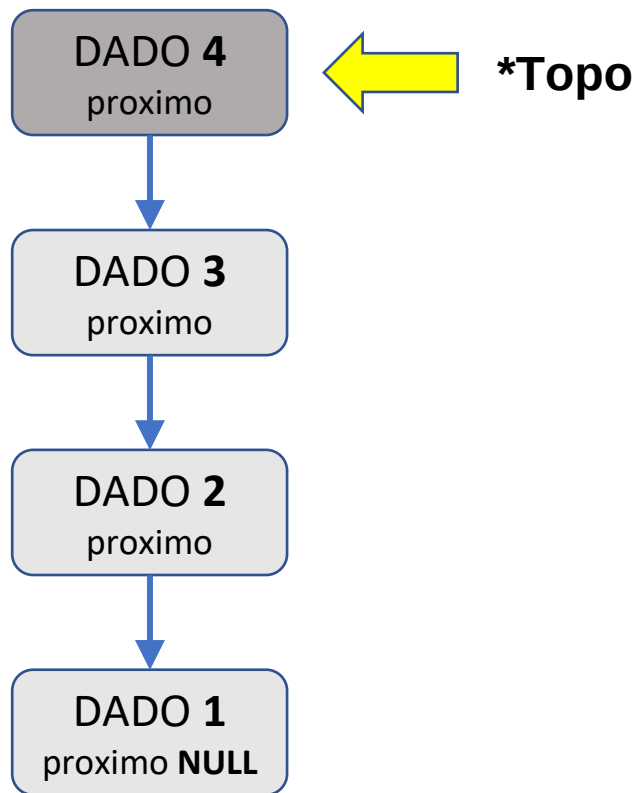
```
typedef struct{  
    char nome[50];  
    char sexo;  
    char cpf[15];  
}Pessoa;
```

```
typedef struct no{  
    char cargo[30];  
    float salario;  
    Pessoa p;  
    struct no *proximo;  
}No;
```

Podemos ter várias informações dentro do nosso **nó**, inclusive outras estruturas (Pessoa no exemplo acima).

Um exemplo prático





Digamos que os dados da pilha seja composta apenas por um valor do tipo **int**.

- Uma pilha é composta de vários **nós** encadeados.
- Cada **nó** possui dois campos:
 - um campo dado
 - um campo ponteiro apontando para o **nó** seguinte.



```
1 //professor moreno
2 #include <stdlib.h>
3 #include <stdio.h>
4
5 void push(int item); //empilhar
6 void pop();          //desempilhar
7 void imprimir();
8 int entrada_dados();
9
10 typedef struct apelido_no{
11     int dado;
12     struct apelido_no *proximo;
13 }no;
14
15 no *top=NULL;
16 //-----
```

Estrutura do programa

```
17 int main(){
18     int n,opcao;
19     do{
20         system("cls");
21         printf("\n\nMenu\n1. Empilhar \n2. Desempilhar\n3. Imprimir \n4. Sair");
22         printf("\nEscolha uma opcao (0-4): ");
23         scanf("%d",&opcao);
24         switch (opcao){
25             case 1:
26                 n=entrada_dados();
27                 push(n); //empilhar
28                 break;
29             case 2:
30                 pop(); //desempilhar
31                 break;
32             case 3:
33                 imprimir();
34                 break;
35         }
36     }while (opcao!=4);
37     system("pause");
38     return 0;}
39 //-----
```

```
40 void push(int item){
41     printf("\nEMPILHANDO...\n");
42     no *novo=malloc(sizeof(no));
43     //necessário verificar se há memória
44     novo->dado=item;
45     novo->proximo=top;
46     top=novo;
47     printf("\nValor %d empilhado\n",novo->dado);
48     system("pause");
49 }
50 //-----
51 void pop(){
52     printf("\nDESEMPILHANDO...\n");
53     if (top==NULL)
54         printf("A pilha esta vazia\n");
55     else{
56         no *temp;
57         temp=top;
58         top=top->proximo;
59         printf("\n%d Desempilhado com sucesso\n",temp->dado);
60         free(temp);
61     }
62     system("pause");
63 }
```



```
65 void imprimir(){
66     no *temp;
67     temp=top;
68     printf("\nIMPRIMINDO PILHA...\n");
69     while (temp!=NULL){
70         printf("%d\n", temp->dado);
71         temp=temp->proximo;
72     }
73     system("pause");
74 }
75 //-----
76 int entrada_dados(){
77     int valor;
78     printf("\nEntre com valor a empilhar: ");
79     scanf("%d",&valor);
80     return valor;
81 }
```

A pilha é composta de vários nós encadeados.

- campo de dado
- campo ponteiro

```
typedef struct apelido_no{  
    int dado;  
    struct apelido_no *proximo;  
}no;
```

```
no *top=NULL;
```

A função **entrada_dados** será útil para inserir dados na pilha.

```
int entrada_dados(){  
    int valor;  
    printf("\nEntre com valor a empilhar: ");  
    scanf("%d",&valor);  
    return valor;  
}
```

- O procedimento **push** irá empilhar um novo dado que foi passado por parâmetro (**item**)

```
void push(int item){  
    printf("\nEMPILHANDO...\n");  
    no *novo=malloc(sizeof(no));  
    //necessário verificar se há memória  
    novo->dado=item;  
    novo->proximo=top;  
    top=novo;  
    printf("\nValor %d empilhado\n",novo->dado);  
    system("pause");  
}
```

- As operações sempre ocorrem no topo.
- O procedimento **push** recebe o topo atual.
- Aloca memória pra um novo nó.
- Atribui o dado recebido (parâmetro).
- O novo nó passa a apontar para o topo da pilha.

```
void push(int item){  
    printf("\nEMPILHANDO...\n");  
    no *novo=malloc(sizeof(no));  
    //necessário verificar se há memória  
    novo->dado=item;  
    novo->proximo=top;  
    top=novo;  
    printf("\nValor %d empilhado\n",novo->dado);  
    system("pause");  
}
```

Item 30

*top=NULL

```
void push(int item){  
    printf("\nEMPILHANDO...\n");  
    no *novo=malloc(sizeof(no));  
    //necessário verificar se há memória  
    novo->dado=item;  
    novo->proximo=top;  
    top=novo;  
    printf("\nValor %d empilhado\n",novo->dado);  
    system("pause");  
}
```

Item
30

*top=NULL

```
void push(int item){
    printf("\nEMPILHANDO...\n");
    no *novo=malloc(sizeof(no));
    //necessário verificar se há memória
    novo->dado=item;
    novo->proximo=top;
    top=novo;
    printf("\nValor %d empilhado\n",novo->dado);
    system("pause");
}
```

dado:
prox:

Item
30

*top=NULL

```
void push(int item){
    printf("\nEMPILHANDO...\n");
    no *novo=malloc(sizeof(no));
    //necessário verificar se há memória
    novo->dado=item;
    novo->proximo=top;
    top=novo;
    printf("\nValor %d empilhado\n",novo->dado);
    system("pause");
}
```

dado: 30

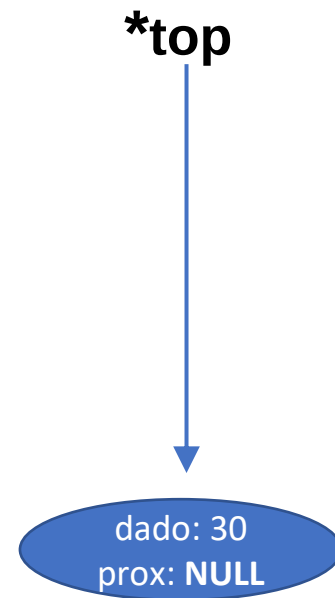
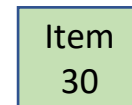
Item
30

*top=NULL

```
void push(int item){
    printf("\nEMPILHANDO...\n");
    no *novo=malloc(sizeof(no));
    //necessário verificar se há memória
    novo->dado=item;
    novo->proximo=top;
    top=novo;
    printf("\nValor %d empilhado\n",novo->dado);
    system("pause");
}
```

dado:30
prox: NULL

```
void push(int item){
    printf("\nEMPILHANDO...\n");
    no *novo=malloc(sizeof(no));
    //necessário verificar se há memória
    novo->dado=item;
    novo->proximo=top;
    top=novo;
    printf("\nValor %d empilhado\n",novo->dado);
    system("pause");
}
```



```
void push(int item){
    printf("\nEMPILHANDO...\n");
    no *novo=malloc(sizeof(no));
    //necessário verificar se há memória
    novo->dado=item;
    novo->proximo=top;
    top=novo;
    printf("\nValor %d empilhado\n",novo->dado);
    system("pause");
}
```

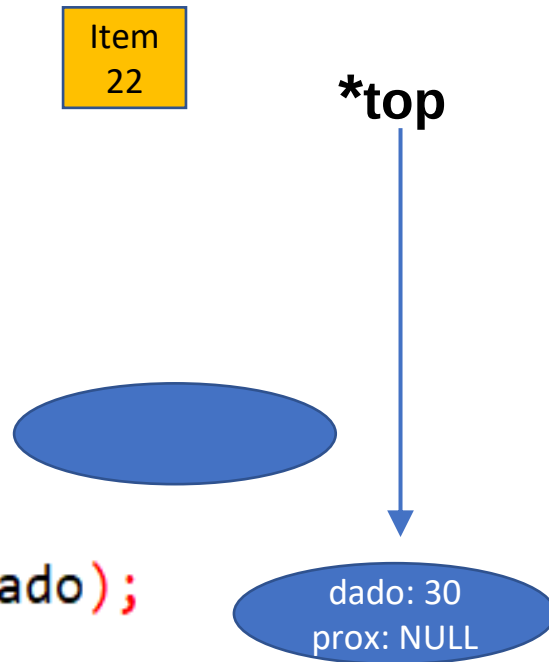
Item
22

***top**

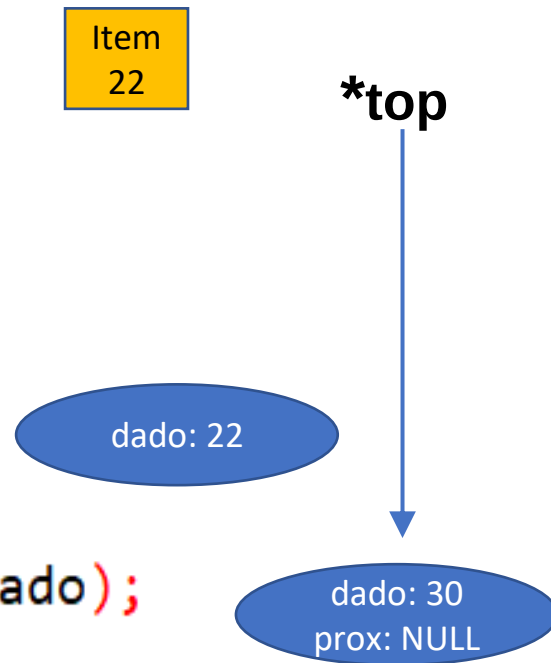


dado: 30
prox: NULL

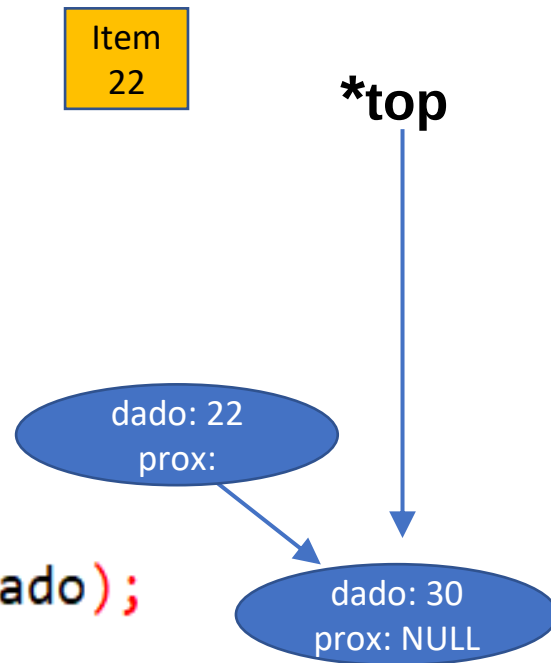
```
void push(int item){
    printf("\nEMPILHANDO...\n");
    no *novo=malloc(sizeof(no));
    //necessário verificar se há memória
    novo->dado=item;
    novo->proximo=top;
    top=novo;
    printf("\nValor %d empilhado\n",novo->dado);
    system("pause");
}
```



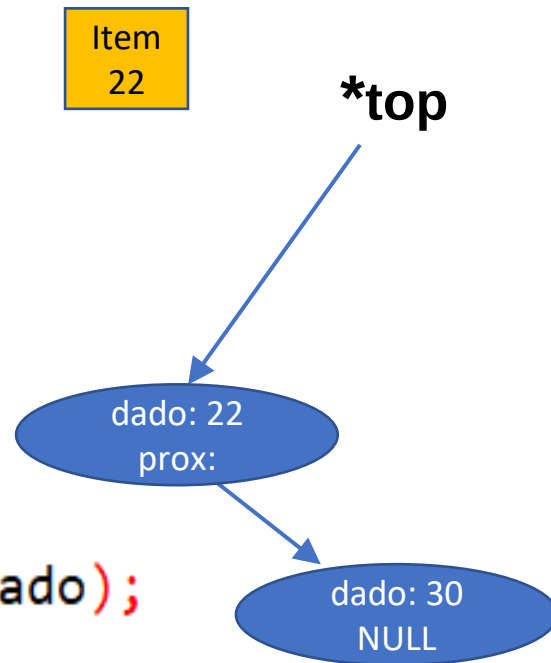
```
void push(int item){
    printf("\nEMPILHANDO...\n");
    no *novo=malloc(sizeof(no));
    //necessário verificar se há memória
    novo->dado=item;
    novo->proximo=top;
    top=novo;
    printf("\nValor %d empilhado\n",novo->dado);
    system("pause");
}
```



```
void push(int item){
    printf("\nEMPILHANDO...\n");
    no *novo=malloc(sizeof(no));
    //necessário verificar se há memória
    novo->dado=item;
    novo->proximo=top;
    top=novo;
    printf("\nValor %d empilhado\n",novo->dado);
    system("pause");
}
```



```
void push(int item){
    printf("\nEMPILHANDO...\n");
    no *novo=malloc(sizeof(no));
    //necessário verificar se há memória
    novo->dado=item;
    novo->proximo=top;
    top=novo;
    printf("\nValor %d empilhado\n",novo->dado);
    system("pause");
}
```





Como
desempilhar um
nó da pilha?

- A operação **pop**.
- Desempilhar (remover) um elemento da pilha, altera o topo da pilha.
- Altera o conteúdo de uma variável ponteiro.


```
void pop(){  
    printf("\nDESEMPILHANDO...\n");  
    if (top==NULL)  
        printf("A pilha esta vazia\n");  
    else{  
        no *temp;  
        temp=top;  
        top=top->proximo;  
        printf("\n%d Desempilhado com sucesso\n",temp->dado);  
        free(temp);  
    }  
    system("pause");  
}
```

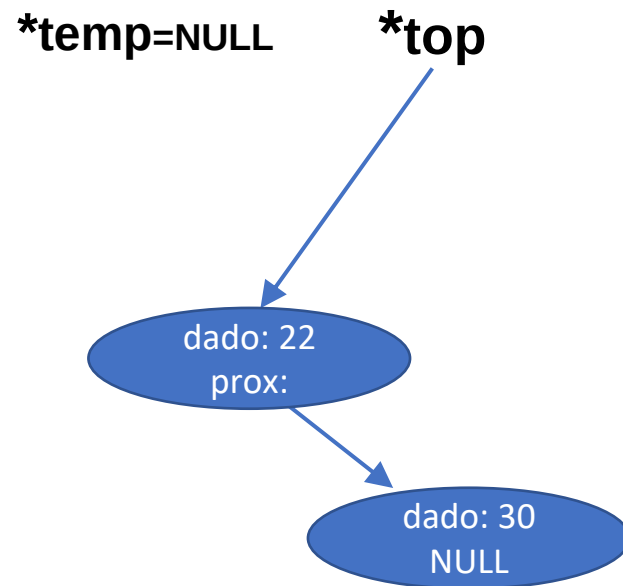
Se a pilha estiver vazia, o conteúdo do topo será NULL.

Caso contrário:

- o ponteiro **temp** recebe o endereço do topo da pilha
- altera o topo
- limpa a memória

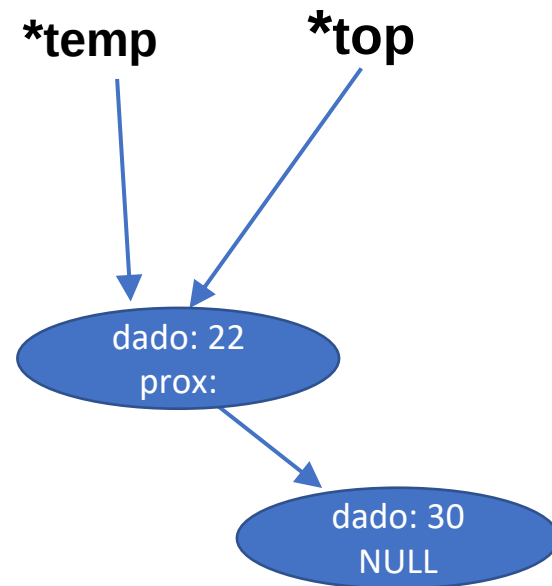
```

void pop(){
    printf("\nDESEMPILHANDO...\n");
    if (top==NULL)
        printf("A pilha esta vazia\n");
    else{
        no *temp;
        temp=top;
        top=top->proximo;
        printf("\n%d Desempilhado com sucesso\n",temp->dado);
        free(temp);
    }
    system("pause");
}
    
```



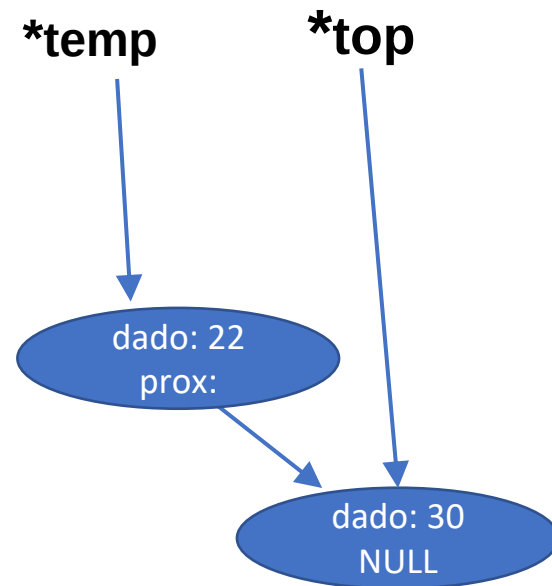
```

void pop(){
    printf("\nDESEMPILHANDO...\n");
    if (top==NULL)
        printf("A pilha esta vazia\n");
    else{
        no *temp;
        temp=top;
        top=top->proximo;
        printf("\n%d Desempilhado com sucesso\n",temp->dado);
        free(temp);
    }
    system("pause");
}
    
```



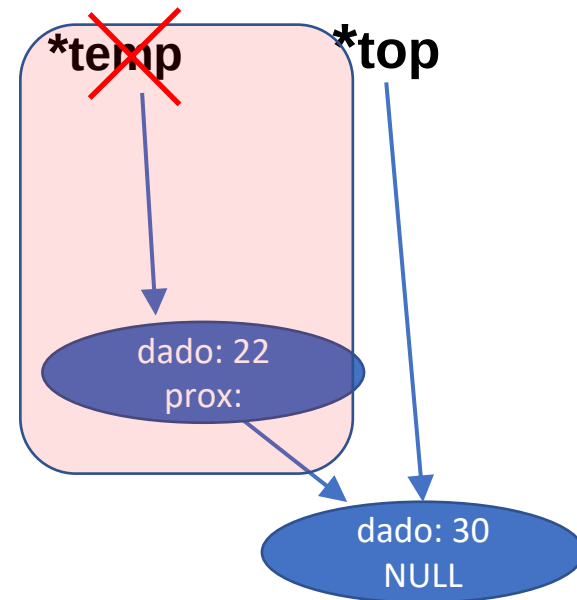
```

void pop(){
    printf("\nDESEMPILHANDO...\n");
    if (top==NULL)
        printf("A pilha esta vazia\n");
    else{
        no *temp;
        temp=top;
        top=temp->proximo;
        printf("\n%d Desempilhado com sucesso\n",temp->dado);
        free(temp);
    }
    system("pause");
}
    
```



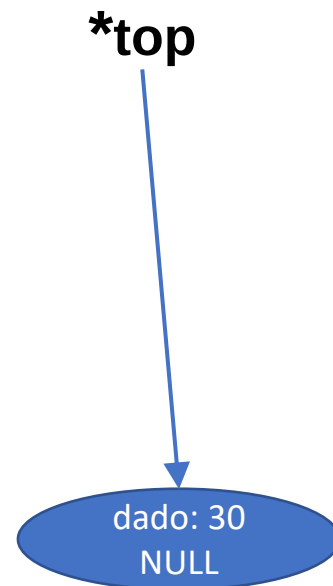
```

void pop(){
    printf("\nDESEMPILHANDO...\n");
    if (top==NULL)
        printf("A pilha esta vazia\n");
    else{
        no *temp;
        temp=top;
        top=temp->proximo;
        printf("\n%d Desempilhado com sucesso\n",temp->dado);
        free(temp);
    }
    system("pause");
}
    
```



```

void pop(){
    printf("\nDESEMPILHANDO...\n");
    if (top==NULL)
        printf("A pilha esta vazia\n");
    else{
        no *temp;
        temp=top;
        top=top->proximo;
        printf("\n%d Desempilhado com sucesso\n",temp->dado);
        free(temp);
    }
    system("pause");
}
    
```





Como apresentar
a pilha?

- Define um ponteiro (temp) tipo “no” e o atribui o topo
- enquanto temp for diferente de NULL, apresenta os dados
- temp aponta para o próximo.

- Define um ponteiro (temp) tipo “no” e o atribui o topo
- enquanto temp for diferente de NULL, apresenta os dados
- temp aponta para o próximo.

```
void imprimir(){  
    no *temp;  
    temp=top;  
    printf("\nIMPRIMINDO PILHA...\n");  
    while (temp!=NULL){  
        printf("%d\n", temp->dado);  
        temp=temp->proximo;  
    }  
    system("pause");  
}
```

SUGESTÃO:
Implemente a
verificação da pilha
estar vazia!

E como fica o
programa
principal?



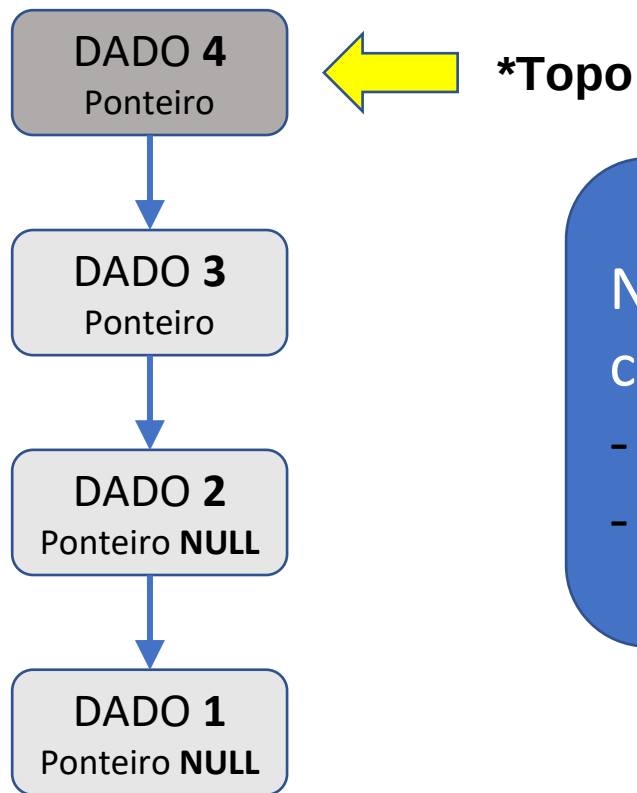
```
int main(){
    int n,opcao;
    do{
        system("cls");
        printf("\n\nMenu\n1. Empilhar \n2. Desempilhar\n3. Imprimir \n4. Sair");
        printf("\nEscolha uma opcao (0-4): ");
        scanf("%d",&opcao);
        switch (opcao){
            case 1:
                n=entrada_dados();
                push(n); //empilhar
                break;
            case 2:
                pop(); //desempilhar
                break;
            case 3:
                imprimir();
                break;
        }
    }while (opcao!=4);
    system("pause");
    return 0;}
```



Exemplo2

(ponteiro para ponteiro)





Nesse segundo exemplo, o dado é composto por:

- Nome
- Data de nascimento

- Uma pilha é composta de vários **nós** encadeados.
- Cada **nó** possuirá dois campos:
 - um campo dado
 - um campo ponteiro apontando para o **nó** seguinte.



```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct{
5     int dia, mes, ano;
6 }Data;
7
8 typedef struct{
9     char nome[30];
10     Data data;
11 }Pessoa;
```

- Implementar o tipo pessoa.
 - nome
 - data de nascimento

Perceba que temos uma struct dentro de outra (Pessoa possui Data)

```
13 typedef struct no{  
14     Pessoa p;  
15     struct no *proximo;  
16 }No;
```

- A pilha é composta de vários nós encadeados.
 - campo de dado (**Pessoa**)
 - campo ponteiro (**proximo**)

```
18 Pessoa ler_pessoa( ){
19     Pessoa p;
20     printf( "\nNome: " );
21     scanf( "%29[^\n]", &p.nome );
22     printf( "\nData de nascimento: " );
23     scanf( "%d%d%d", &p.data.dia,
&p.data.mes, &p.data.ano );
24     return p;
25 }
```

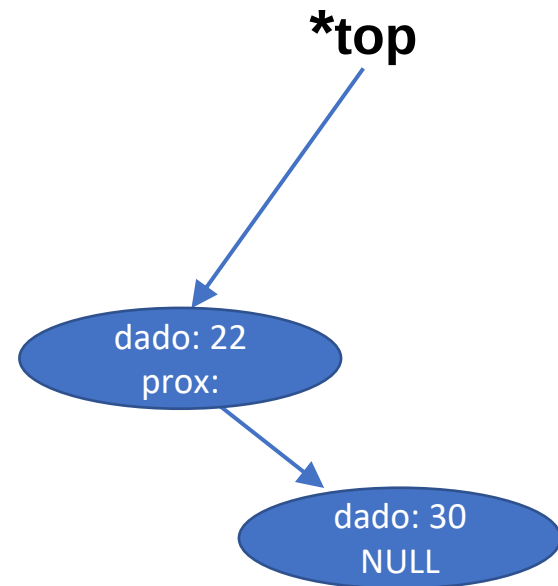
- Função para ler as informações de uma pessoa.
- Útil para inserir uma pessoa na pilha.


```
31 ~ No* empilhar(No *topo){  
32     No *novo = malloc(sizeof(No));  
33  
34 ~     if(novo){  
35         novo->p = ler_pessoa();  
36         novo->proximo = topo;  
37         return novo;  
38     }else  
39         printf("\nErro ao alocar  
memória.\n");  
40     return NULL;  
41 }
```

- As operações sempre ocorrem no topo.
- A função inserir recebe o topo atual.
- Aloca memória pra um novo nó.
- Lê os dados de uma pessoa.
- O novo nó passa a apontar para o topo da pilha.

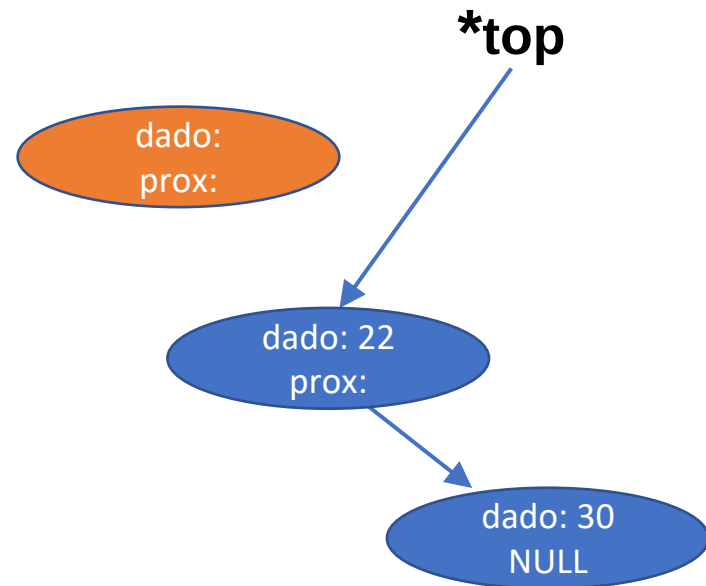
```

31 ~ No* empilhar(No *topo){
32     No *novo = malloc(sizeof(No));
33
34 ~     if(novo){
35         novo->p = ler_pessoa();
36         novo->proximo = topo;
37         return novo;
38     }else
39         printf("\nErro ao alocar
memória.\n");
40     return NULL;
41 }
    
```



```

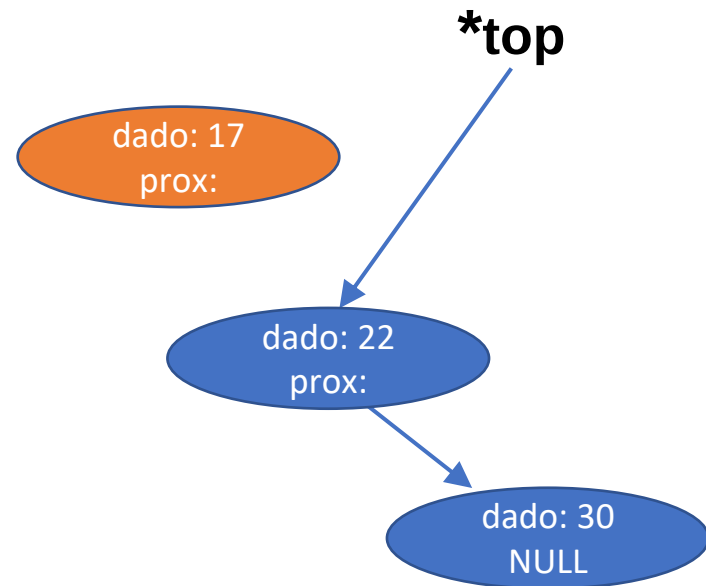
31 ~ No* empilhar(No *topo){
32     No *novo = malloc(sizeof(No));
33
34 ~     if(novo){
35         novo->p = ler_pessoa();
36         novo->proximo = topo;
37         return novo;
38     }else
39         printf("\nErro ao alocar
memória.\n");
40     return NULL;
41 }
    
```



```

31 ~ No* empilhar(No *topo){
32     No *novo = malloc(sizeof(No));
33
34 ~     if(novo){
35         novo->p = ler_pessoa();
36         novo->proximo = topo;
37         return novo;
38     }else
39         printf("\nErro ao alocar
memória.\n");
40     return NULL;
41 }

```

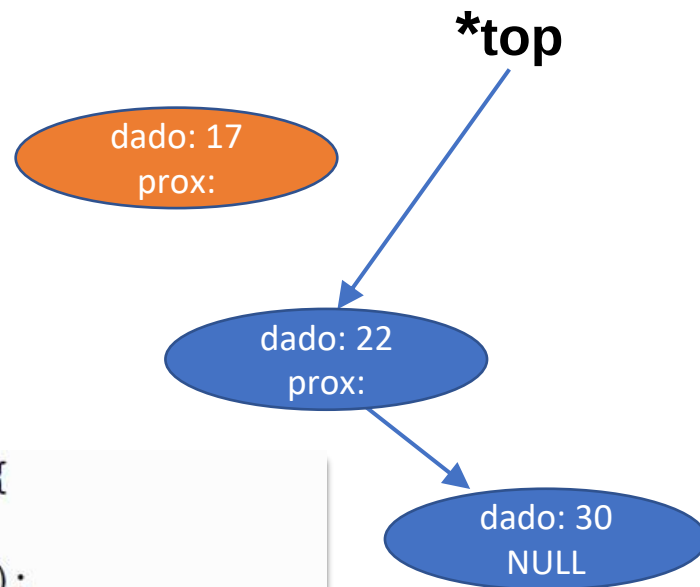


```

31 ~ No* empilhar(No *topo){
32     No *novo = malloc(sizeof(No));
33
34 ~     if(novo){
35         novo->p = ler_pessoa();
36         novo->proximo = topo;
37         return novo;
38     }else
39         printf("\nErro ao
memória.\n");
40     return NULL;
41 }
    
```

```

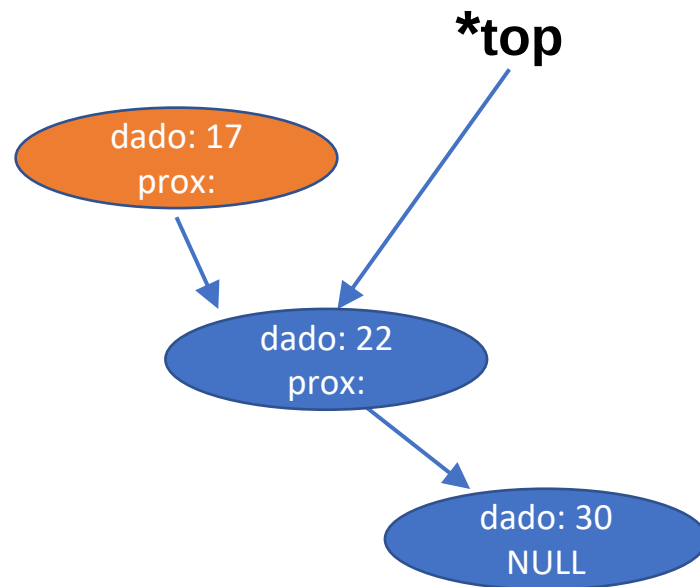
Pessoa ler_pessoa(){
    Pessoa p;
    printf("\nNome: ");
    scanf("%29[^\n]", &p.nome);
    printf("\nData de nascimento: ");
    scanf("%d%d%d", &p.data.dia,
    &p.data.mes, &p.data.ano);
    return p;
}
    
```



```

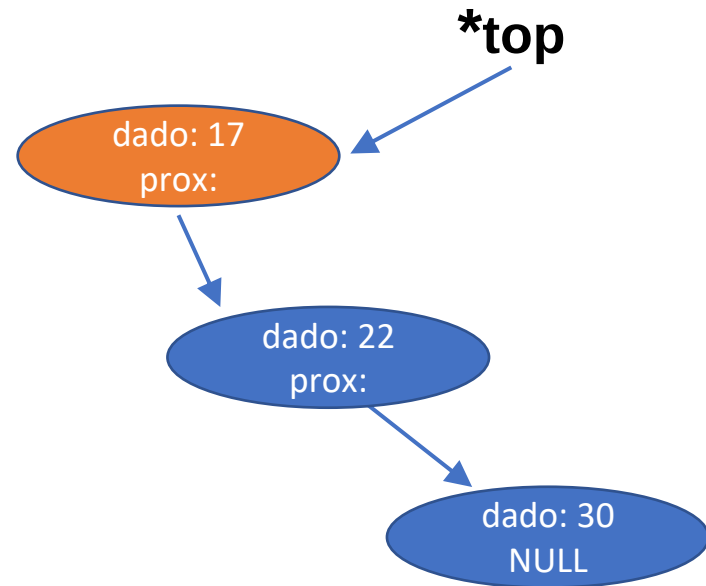
31 ~ No* empilhar(No *topo){
32     No *novo = malloc(sizeof(No));
33
34 ~     if(novo){
35         novo->p = ler_pessoa();
36         novo->proximo = topo;
37         return novo;
38     }else
39         printf("\nErro ao alocar
memória.\n");
40     return NULL;
41 }

```



```

31 ~ No* empilhar(No *topo){
32     No *novo = malloc(sizeof(No));
33
34 ~     if(novo){
35         novo->p = ler_pessoa();
36         novo->proximo = topo;
37         return novo;
38     }else
39         printf("\nErro ao alocar
memória.\n");
40     return NULL;
41 }
    
```



```

switch(opcao){
case 1:
    topo = empilhar(topo);
    break;
}
    
```

```
27 void imprimir(Pessoa p){  
28     printf("\nNome: %s\nData:  
    %2d/%2d/%4d", p.nome, p.data.dia,  
    p.data.mes, p.data.ano);  
29 }
```

- Procedimento para imprimir as informações de uma pessoa.

Como
desempilhar um
nó da pilha?



- A operação **pop**.
- Desempilhar (remover) um elemento da pilha, altera o topo da pilha.
- Altera o conteúdo de uma variável ponteiro.

- Retorna o ponteiro para o nó que será removido.
- A alteração precisa ser feita dentro da função.

Como alterar o conteúdo
de uma variável declarada
em outra função?



- A função recebe como parâmetro um **ponteiro para ponteiro**.
- O endereço da variável declarada na função main que contém o **endereço do topo da pilha**.

- Se a pilha estiver vazia o conteúdo do topo é NULL.
- Caso contrário
 - o ponteiro remover recebe o endereço do topo da pilha
 - altera o topo
 - retorna o ponteiro

```
43 ~ No* desempilhar(No **topo){  
44 ~     if (*topo != NULL){  
45         No *remover = *topo;  
46         *topo = remover->proximo;  
47         return remover;  
48     }else  
49         printf("\nPilha vazia.\n");  
50     return NULL;  
51 }
```

```
case 2:  
    remover = desempilhar(&topo);  
    if(remover){  
        printf("\nElemento removido.\n");  
        imprimir(remover->p);  
        free(remover);
```

Imprimir a pilha

- Recebe o ponteiro para o topo da pilha.
- Enquanto o topo for diferente de NULL
 - imprime as informações de cada pessoa.



```
53 void imprimir_pilha(No *topo){  
54     while(topo){  
55         imprimir(topo->p);  
56         topo = topo->proximo;  
57     }  
58 }
```

```
60 ~ int main(void) {  
61     No *remover, *topo = NULL;  
62     int opcao;  
63  
64 ~     do{  
65         printf("\n0 - Sair\n1 - Empilhar\n2 - Desempilhar\n3 - Imprimir\n");  
66         scanf("%d", &opcao);  
67         getchar();  
68  
69 ~         switch(opcao){  
70             case 1:  
71                 topo = empilhar(topo);  
72                 break;
```

```
73     case 2:
74         remover = desempilhar(&topo);
75         if(remover){
76             printf("\nElemento removido.\n");
77             imprimir(remover->p);
78             free(remover);
79         }else
80             printf("\nElemento não existe.\n");
81         break;
82     case 3:
83         imprimir_pilha(topo);
84         break;
85     default:
86         if(opcao != 0)
87             printf("\nOpção inválida.\n");
88     }
89 }while(opcao != 0);
90 }
```

